



udp UNIVERSIDAD
DIEGO PORTALES

UNIVERSIDAD DIEGO PORTALES
ESCUELA DE INFORMÁTICA &
TELECOMUNICACIONES

LAB 5:

Connect-Four Game

Autores:

Martin Correa

Joakin Mac-Auliffe

Sergio Pinto

Profesor:

Marcos Fantoal

Link de GitHub

Índice

1. Introducción	2
2. Metodología	2
2.1. Clase Player	2
2.2. Clase Scoreboard	3
2.3. Clase ConnectFour	4
2.4. Clase Game	5
2.5. Clase SearchTrees	6
2.6. Clase Main	8
3. Análisis	8
3.1. Análisis Asintótico	8
3.2. Ventajas y Desventajas de la Implementación	9
3.3. Desafíos Encontrados	9
3.4. Soporte para torneos de múltiples jugadores	9
3.5. Revisión de estructura de datos	11
3.6. Prueba interactiva de la implementación	11
4. Conclusión	13

1. Introducción

En el presente informe se desarrollará del juego de mesa ConnectFour (Conecta 4) en Java. Para ello, se hará uso de distintas estructuras de datos acordes a las necesidades que se presenten. Luego, se realizará un análisis asintótico de la eficiencia de los métodos del programa. Además, se documentarán los desafíos presentados y se propondrán mejoras para el programa. Finalmente, se hará uso de la implementación realizada y se realizarán observaciones.

2. Metodología

En esta sección se explicará el funcionamiento del programa, describiendo de forma detallada todas las clases, junto con sus atributos y respectivos métodos.

2.1. Clase Player

■ Descripción

Clase que representa a cada jugador que registra en el juego, considerando atributos como su nombre, partidas ganadas, etc.

■ Atributos

- **String playerName:** Representa el nombre del jugador.
- **int wins:** Contador de las partidas ganadas del jugador.
- **int draws:** Contador de las partidas empatadas del jugador.
- **int losses:** Contador de las partidas perdidas del jugador.

■ Métodos

- **Player(String playerName):** Constructor de la clase *Player*, le asigna valor a los atributos de la clase, según los parámetros entregados. **Complejidad $O(1)$.**
- **void addWin():** Función que incrementa en una unidad el atributo *wins*. **Complejidad $O(1)$.**
- **void addDraw():** Función que incrementa en una unidad el atributo *draws*. **Complejidad $O(1)$. Complejidad $O(1)$.**
- **void addLoss():** Función que incrementa en una unidad el atributo *losses*. **Complejidad $O(1)$.**
- **Getters — Complejidad $O(1)$**
 - String getPlayerName()

-
- `int getWins()`
 - `int getDraws()`
 - `int getLosses()`

2.2. Clase Scoreboard

■ Descripción

Clase que representa el marcador. Su función es gestionar el registro de jugadores y almacenar los resultados de las partidas.

■ Atributos

- **BST winTree:** Objeto de tipo BST (Árbol de búsqueda binaria implementado en el programa) cuya función es ordenar a los jugadores por su número de victorias. La llave es el número de victorias (*wins*) y el valor es el nombre del jugador (*playerName*).
- **HashST players:** Objeto de tipo HashST (Tabla de hash implementada en el programa) cuya función es almacenar a los jugadores asociando su nombre (*playerName*) con su instancia de la clase *Player*.
- **TreeMap<Integer, List<Player>> playersByWins:** TreeMap cuya key es un valor entero que representa un número específico de partidas ganadas (*wins*), y su valor es una lista que contiene a todos los jugadores (instancias de tipo *Player*) cuyo atributo *wins* es igual al de la key. Se utiliza en los métodos *Player[] winRange(int low, int high)* y *Player[] winSuccessor(int wins)*, pues de esta manera se facilita el rastreo de jugadores según su cantidad de partidas ganadas.
- **int playedGames:** Contador que se incrementa cada vez que se registra una nueva partida, de tal manera de mantener registro de la cantidad total de partidas jugadas.

■ Métodos

- **Scoreboard():** Constructor de la clase *Scoreboard*, instancia los atributos de la clase. **Complejidad $O(1)$.**
- **void addGameResult(String winnerPlayerName, String loserPlayerName, boolean draw:)** Función que aumenta en un valor el atributo *playedGames*, además del contador de las partidas ganadas y perdidas de los jugadores. Adicionalmente, ajusta la llave del jugador *WinnerPlayerName* en el map *playersByWins*, puesto que ahora va a pertenecer a la nueva lista correspondiente según sus partidas ganadas. **Complejidad $O(\log n)$,** puesto que se realizan operaciones como *pla-*

yersByWins.get(), *playersByWins.remove()*, las cuales tienen complejidad $O(\log n)$, debido a la naturaleza de un *TreeMap*.

- **void registerPlayer(String playerName):** Registra un nuevo jugador, verificando que no haya sido ingresado previamente. **Complejidad $O(\log n)$** , debido al uso del comando *playersByWins.computeIfAbsent(newPlayer.getWins(), k -> new ArrayList<>()).add(newPlayer)* el cual tiene esa complejidad.
- **int getPlayedGames():** Retorna la cantidad de partidas jugadas. **Complejidad $O(1)$** .
- **boolean checkPlayer(String playerName):** Verifica la existencia de algún jugador ingresado previamente con ese nombre. **Complejidad $O(1)$** , puesto que realizar una búsqueda en una tabla de hash tiene esa complejidad.
- **Player[] winRange(int low, int high):** Función que retorna la lista de jugadores que estén dentro del rango de victorias otorgada por el usuario. **Complejidad $O(n + k)$** , donde n es la cantidad diferente de partidas ganadas, es decir, la cantidad de llaves que tiene el mapa, y k la cantidad de jugadores con ese número de victorias.
- **Player[] winSucesor(int wins):** Retorna la lista de jugadores que tiene exactamente una victoria más que el número ingresado. **Complejidad $O(n + k)$** , donde n es la cantidad diferente de partidas ganadas, es decir, la cantidad de llaves que tiene el mapa, y k la cantidad de jugadores con ese número de victorias.
- **void printScoreboard():** Imprime la tabla de resultados de cada uno de los jugadores. **Complejidad $O(n + p)$** , donde n es la cantidad diferente de partidas ganadas, es decir, la cantidad de llaves que tiene el mapa, y p la cantidad de jugadores totales en todo el juego.
- **Player getPlayer(String name):** Retorna al jugador cuyo nombre coincida con el parámetro *name* que esté en la tabla de hash *players*. De no existir un jugador con dicho nombre, retorna *null*.

2.3. Clase ConnectFour

■ Descripción

Clase que se encarga del funcionamiento del juego, creando el tablero donde se jugará, además de realizar los movimientos que soliciten.

■ Atributos

- **char[][] grid = new char[7][6]:** Tablero donde se llevará a cabo el juego.

-
- **char currentSymbol**: Jugador que se encuentra en turno.

- **Métodos**

- **ConnectFour()**: Constructor de la clase, inicializa todos los atributos de la clase, rellenando el tablero con espacios y asignándole el valor x al jugador en turno. **Complejidad $O(n * m)$** , debido a que recorre la matriz *grid* en su totalidad, para inicializar sus valores en ' '.
- **Boolean makeMove(int z)**: Función encargado de posicionar el símbolo del jugador en turno en la columna proporcionada, verificando que haya espacio disponible. **Complejidad $O(n)$** , donde n corresponde al tamaño de las columnas.
- **char isGameOver()**: Método cuya función es verificar el estado del juego, es decir, si algún jugador ganó, si terminó en empate o si continúa. **Complejidad $O(n * m)$** , debido que para verificar si algún jugador logró ganar, se recorre la matriz de forma completa.
- **void printBoard()**: Función que imprime el tablero. **Complejidad $O(n * m)$** .
- **void clearBoard()**: Método que limpia todo el tablero para el siguiente juego. **Complejidad $O(n * m)$** .
- **Setters, Getters — Complejidad $O(1)$**
 - **getCurrentSymbol()**
 - **setCurrentSymbol()**

Estos métodos no son estrictamente necesarios, sin embargo permiten que el programa funcione mejor.

2.4. Clase Game

- **Descripción**

Clase que tiene como fin representar una partida del juego conecta 4 entre dos jugadores.

- **Atributos**

- **String status**: Variable de tipo String que guarda e informa acerca del estado del juego.
- **String winnerPlayerName**: Variable de tipo String que almacena el nombre del ganador de la partida.
- **String playerNameA**: Variable de tipo String que almacena el nombre del jugador A.

- **String playerNameB:** Variable de tipo `String` que almacena el nombre del jugador B.
- **ConnectFour connectFour:** Objeto de tipo `ConnectFour`, que permitirá el funcionamiento del juego.
- **Scoreboard scoreboard:** Objeto de tipo `Scoreboard`, que servirá para guardar el puntaje como un marcador.
- **int turn:** Variable de tipo `int` que servirá como contador de turnos.

■ Métodos

- **Game(String playerNameA, String playerNameB):** Constructor que inicializa una partida nueva con los jugadores A y B pedidos por el constructor. También inicializa el estado de la partida en `"IN_PROGRESS"`. **Complejidad $O(1)$.**
- **String play():** Método que tiene como finalidad ejecutar tantas partidas como el usuario desee del juego "Connect Four". Esto se realiza utilizando un ciclo, el cual, en cada iteración llama al método `makeMove(int z)` e `isGameOver()` de la clase `ConnectFour`, para realizar el movimiento del jugador y verificar si alguno de los jugadores ganó. El juego termina una vez que haya un ganador o el tablero se llene, lo que culminaría en empate. **Complejidad $O(p * (n * m))$** , donde p corresponde a la cantidad de movimientos que se realizan hasta que haya un ganador o hasta que termine en empate, y $(n * m)$, dado que se llaman las funciones `makeMove(int z)` y `isGameOver()`, las cuales tienen esa complejidad.
- **Getters — $O(1)$**
 - `getConnectFour()`
 - `getScoreboard()`

2.5. Clase SearchTrees

• Descripción

Clases donde van a ser creadas las listas o métodos de ordenamiento, como BST, HashSet, Nodos, que se ocuparan dentro del código.

• Atributos Clase Node

- **K key:** Representa la llave con la que será identificado el nodo.
- **V value:** El valor que almacena el nodo.
- **Node<K, V> left, right** Nodos situados a la izquierda (menores) y derecha (mayores), con respecto al nodo actual.

-
- **Atributos Clase BST**
 - **Node<Integer, String> root:** Nodo raíz del árbol.
 - **Atributos Clase HashST**
 - **int CAPACITY:** Tamaño de la tabla de hash, la cual corresponde a 16.
 - **Node<String, Player>[] table:** Un arreglo de nodos, donde cada nodo almacenara un String y objeto Player.
 - **Métodos Clase Node**
 - **Node(K key, V value):** Constructor que da valor a key y a value. Complejidad $O(1)$.
 - **Getters — Complejidad $O(1)$**
 - ◊ **K getKey()**
 - ◊ **V getValue()**
 - ◊ **Node<K, V> getLeft()**
 - ◊ **Node<K, V> getRight()**
 - **Metodos Clase BST**
 - **Node<Integer, String> insertRec(Node<Integer, String> node, int key, String value):** Método que inserta un nuevo nodo de manera recursiva al árbol binario con el fin de ordenar a los jugadores por numero de victorias. **Complejidad $O(h)$** , donde h corresponde a la altura del árbol.
 - **void insert(Player P):** Método que tiene como fin insertar al jugador "P" en el árbol de *root*, llamando a la función *insertRec()*. **Complejidad $O(h)$** .
 - **String get(int key):** Método que busca al jugador asociado a la key insertada. **Complejidad $O(h)$** .
 - **Node<Integer, String> getRoot:** Método que retorna la raíz del árbol. **Complejidad $O(1)$** .
 - **Métodos Clase HashST**
 - **HashST():** Constructor que inicializa el arreglo de nodos *table*. **Complejidad $O(1)$** .
 - **int hash(String key):** Retorna un índice compatible para la tabla de hash basado según la clave *key*. **Complejidad $O(1)$** .

-
- **void insert(Player p):** Método utilizado para insertar a un jugador "P" dentro de la tabla de hash. **Complejidad $O(1)$.**
 - **Player get(String key):** Método que retorna al jugador que se encuentra en *key*, en caso de no encontrarlo retorna *null*. **Complejidad $O(1)$.**

2.6. Clase Main

■ Descripción

Clase donde se ejecuta el código. Se inicializa la clase *Game*, los jugadores ingresan sus nombres y la cantidad de partidas que jugarán, luego se llama a la función *play()* para comenzar el juego.

3. Análisis

3.1. Análisis Asintótico

- **addGameResult:** El tiempo de ejecución de este método varía dependiendo de si hay empate, victoria/derrota y si hay victoria/derrota pero el árbol binario se encuentra desbalanceado. Si es que existe un empate el tiempo total es de $O(1)$ ya que se hace un *.addDraw()* a ambos jugadores y el método termina. En cambio si es victoria/derrota el tiempo de ejecución es de $O(\log n + m + \log j)$ donde *n* es el número de claves, *m* es la cantidad de jugadores con el mismo número de victorias y *j* es el número total de jugadores, en caso de que el árbol binario este balanceado y $O(m + \log n + j)$ en caso de que el árbol se encuentre desbalanceado.
- **registerPlayer:** El tiempo de ejecución promedio de este método es de $O(\log c)$ donde *c* es la cantidad de claves en el *TreeMap* "playersByWins". En el peor de los casos el tiempo de ejecución sería de $O(n)$, esto solo pasaría si al momento de registrar jugadores existen demasiadas colisiones.
- **checkPlayer:** El tiempo de ejecución promedio es de $O(1)$ ya que se utiliza una tabla de hash. Si es que llegan a existir demasiadas colisiones entonces el tiempo de ejecución aumenta a $O(N)$
- **winRange:** La complejidad total del método va a ser de $O(\log c + j)$ donde *c* indica el número total de claves en el *TreeMap* "playersByWins" y *j* indica el número total de jugadores.
- **winSucesor:** En este método la complejidad total va a ser de $O(\log c + j)$ donde *c* es el número de claves distintas dentro del *TreeMap* "playersByWins" y *j* es la cantidad de jugadores con más victorias que la variable "wins". $O(\log$

c) es provocado cuando se ocupa `.higherEntry(wins)`. El `O(j)` es provocado cuando se utiliza `.toArray(new Player[0])`.

3.2. Ventajas y Desventajas de la Implementación

Entre las ventajas se destaca la simplicidad de la implementación, pues para jugar no se requiere más que un compilador de java y un computador con al menos de 400 kilobytes de espacio de almacenamiento disponible en el disco duro. También, una gran ventaja de la implementación es la facilidad de su ejecución, pues todo el funcionamiento interior del programa se administra mediante una sola instancia de la clase *Game*, así simplificando mucho la planificación de las partidas, al tener que utilizar una sola función para ejecutar el programa completo. Pese a la conveniencia de esto último, una desventaja de la implementación es que el juego se limita a sólo dos jugadores, pues para incluir más jugadores se tendrían que crear múltiples instancias de la clase *Game* y cada una tendría un scoreboard distinto, por lo que se volvería mucho menos intuitivo elegir a un ganador y un perdedor. La otra gran desventaja que tiene esta implementación tiene es que los datos recolectados durante los juegos no se almacenan en ninguna parte, por lo que para realizar estadísticas habría que registrar todos los datos a mano.

3.3. Desafíos Encontrados

Lo que más se dificultó en el desarrollo de este laboratorio fue la implementación de la clase *HashST*, puesto que es la estructura de datos vista más recientemente en clases, debido a su complejidad considerable fue lo más difícil de implementar en el código. Con la ayuda de apuntes e información otorgada por el profesor, sumado a videos de internet, se pudo entender de mejor manera la forma correcta con la que debía estar implementada la tabla de hash, adaptándose al contexto del código.

3.4. Soporte para torneos de múltiples jugadores

Para que el programa posea la capacidad de realizar torneos de varios jugadores, se propone la siguiente reestructuración del programa:

El constructor de la clase *Game* en vez de recibir las variables tipo *String* *playerNameA* y *playerNameB*, pasaría a recibir como parámetro una lista de *Strings* cuyo contenido corresponde a los nombres de los jugadores que participarán. Posteriormente, dichos nombres se registrarán en el Scoreboard mediante un ciclo *for* que ejecutará el método *registerPlayer()* utilizando como parámetro cada elemento de la lista previamente ingresada como parámetro del constructor. Finalmente, la lista de nombres se agregará como atributo a la clase, pues será utilizada posteriormente en el método *play()*. Dicho atributo funcionará asignando la posición de cada jugador en la lista más uno como su identificador ($n + 1$), de la siguiente manera:

```
List<String>> playersByIndex = {"playerA", "playerB", "playerC"}
```

Donde playerA tendría el índice 1, playerB el índice 2 y playerC el índice 3.

En el método *play()*, primero se solicitará al usuario que ingrese dos números que representen la disposición de las partidas, de la siguiente manera (siendo n el número de participantes en el torneo):

- De haber un número **par** de jugadores, se ejecuta un ciclo *for* que itera $n/2$ veces, donde cada iteración representa una partida. En cada iteración, el usuario deberá ingresar los dos jugadores que jugarán en dicha partida, de la siguiente manera: ([índice de jugador 1] [índice de jugador 2]).
- De haber un número **impar** de jugadores, se ejecuta el ciclo *for* de igual manera que cuando hay un número par de jugadores, pero en este caso, el jugador que queda sin asignar juega en la siguiente ronda.

El funcionamiento de las rondas se implementará de manera que todos los jugadores compitan entre ellos, de esta manera generando $\frac{n(n+1)}{2}$ partidas repartidas en n rondas si n es par y $n - 1$ rondas si n es impar, siendo n el número de jugadores. En caso de haber un número impar de jugadores, en cada ronda quedará uno sin jugar.

El algoritmo utilizado para asignar las rondas consiste en un ciclo que itera n veces, siendo n la cantidad de participantes en el torneo. Dicho ciclo empareja a los extremos por cada iteración (Si hay 5 participantes, se empareja al 1 con el 5 y al 2 con el 4), y de ser n impar, el participante del medio no juega hasta la siguiente ronda. Mediante este algoritmo, cada jugador juega contra todos exactamente una vez.

Un ejemplo de una distribución de rondas sería el siguiente, tomando 5 jugadores como ejemplo (A, B, C, D y E):

Cuadro 1: Emparejamientos por ronda (5 jugadores)

Ronda	Partida 1	Partida 2	No juega
1	A vs E	B vs D	C
2	A vs D	C vs E	B
3	A vs C	B vs E	D
4	A vs B	C vs D	E
5	B vs C	D vs E	A

Para determinar quién gana, pierde o empata, se utiliza un sistema de puntos, en base a los cuales se determinan los resultados finales. Dichos puntos se calculan mediante las siguientes condiciones:

- Si el jugador gana una partida, suma **3 puntos**.
- Si el jugador pierde una partida, suma **0 puntos**.
- Si el jugador empata una partida, suma **1 punto**.

Finalmente, se declara ganador al jugador con mayor cantidad de puntos y perdedor al jugador con menor cantidad de puntos. En el caso de que dos jugadores terminen con la misma cantidad de puntos, se juega una partida final entre ambos, en base a la cual se decidirá al ganador.

3.5. Revisión de estructura de datos

Luego de analizar el programa con detenimiento, se llegó a la conclusión de que una alternativa válida para reemplazar *HashST* sería con un *TreeMap*, ya que de igual manera permite identificar los elementos con una llave, la principal ventaja sería que es mucho más fácil y rápida de entender e implementar, sin embargo sería mucho menos eficiente, puesto que al funcionar con un árbol, la mayoría de sus métodos tienen complejidad $O(\log n)$, la cual es mucho mayor a la $O(1)$ que proporciona la tabla de Hash.

El uso de una *Linked-List* podría considerarse como una alternativa válida para reemplazar *BST*, puesto que es una estructura de datos que es fácil de entender y manejar, además funciona con nodos al igual un *BinarySearchTree*, no obstante es menos eficiente, ya que al realizar búsquedas habría que recorrer la *Linked-List* de forma completa, resultando en una complejidad $O(n)$, siendo esta mayor a su contraparte, la cual corresponde a $O(\log n)$.

3.6. Prueba interactiva de la implementación

Para asegurar el correcto funcionamiento del programa, se realizaron pruebas utilizando todas las funciones implementadas. Las pruebas consistieron en 29 partidas jugadas entre tres jugadores, en este caso Sergio, Joakin y Martin.

Los resultados de las partidas son los siguientes:

Jugador	Victorias	Derrotas	Empates
Joakin	13	3	4
Martin	9	7	4
Sergio	3	15	0

- **Siguiente jugador con más de 8 victorias:** Martin
- **Jugadores con un rango de victorias entre 6 y 14:** Martin, Joakin
- **Total de partidas jugadas:** 29
- **Tasas de victorias:**
 - Sergio: 0.16
 - Joakin: 0.65
 - Martin: 0.45

```
| | | | | | | |
| | | | | | | |
|x| | | | | | |
|x|o| | | | | |
|x|o| | | | | |
|x|o| | | | | |
  1 2 3 4 5 6 7
El jugador Martin (x) gana la partida!
```

Figura 1: Ejemplo de victoria

```
|x|o|x|o|x|o|o|
|x|o|x|o|x|o|x|
|o|x|o|x|o|x|o|
|o|x|o|x|o|x|o|
|x|o|x|o|x|o|x|
|x|o|x|o|x|o|x|
  1 2 3 4 5 6 7
Empate.
```

Figura 2: Ejemplo de empate

```
--x==x==x== SCOREBOARD --x==x==x==  
Joakin | Victorias: 13 | Derrotas: 3 | Empates: 4  
Martin | Victorias: 9 | Derrotas: 7 | Empates: 4  
Sergio | Victorias: 3 | Derrotas: 15 | Empates: 0  
Siguiente jugador con más de 8 victorias: Martin  
Jugadores con un rango de victorias entre 6 y 14: Martin Joakin  
Total de partidas jugadas: 29  
Tasa de victorias de Sergio: 0.16  
Tasa de victorias de Joakin: 0.65  
Tasa de victorias de Martin: 0.45
```

Figura 3: Ejemplo de scoreboard en la consola

4. Conclusión

Finalmente, se puede observar que se cumplieron los objetivos planteados, utilizando de manera exitosa las estructuras de datos requeridas. Se logró implementar un código capaz de ejecutar el juego *Connect-Four* utilizando tablas de hash, árboles binarios, entre otros. El hecho de que el programa funcione de buena manera demuestra que se logró entender y aplicar de forma adecuada los conceptos solicitados.

De igual manera, queda en evidencia la relevancia de estas estructuras de datos en aplicaciones que se utilizan de forma diaria, ya que permiten almacenar grandes cantidades de datos de forma ordenada y con tiempos de acceso rápidos, lo que garantiza una eficiencia considerable en el programa. Este tipo de estructuras son importantes incluso en proyectos a gran escala, tal es el caso de empresas como Google que desarrollan buscadores web, dichos programas requieren de eficiencia a la hora de buscar y almacenar información, por lo que precisan de estructuras de datos con estas características.